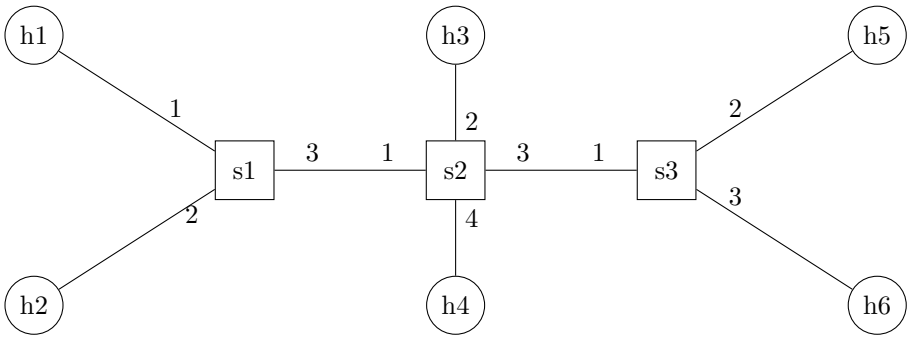
Software Defined Networking – data

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

Nodo → interfaccia: MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

Negli switch sono preinstallate le seguenti regole.

Switch s1		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP1 ip_proto = IPPROTO_TCP tcp_dst = 8080	DecNwTTL Output(1)
5	eth_type = ETH_TYPE_IP ipv4_src = IP1 ip_proto = IPPROTO_TCP tcp_src = 8080	DecNwTTL Output(3)

Switch s2		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP1 ip_proto = IPPROTO_TCP tcp_dst = 80	tcp_dst=8080 DecNwTTL Output(1)
5	eth_type = ETH_TYPE_IP ipv4_src = IP1 ip_proto = IPPROTO_TCP tcp_src = 8080	tcp_src=80 DecNwTTL Output(3)

Switch s3		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP1 ip_proto = IPPROTO_TCP tcp_dst = 80	DecNwTTL Output(1)
5	eth_type = ETH_TYPE_IP ipv4_src = IP1 ip_proto = IPPROTO_TCP tcp_src = 80	DecNwTTL Output(2) Output(3)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (pkt_udp is not None) and (pkt_udp.dst_port==123):
        if (pkt_ipv4.total_length < 114):
            pkt_ipv4.ttl = pkt_ipv4.ttl - 3
            pkt_udp.src_port = pkt_udp.src_port + 1
            pkt.serialize()

            actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
            out = parser.OFPPacketOut(datapath=datapath, data = pkt.data
                in_port=in_port, actions=actions)
            datapath.send_msg(out)
        return

    if (datapath.id == 2) and (pkt_tcp is not None) and (pkt_ipv4.dst==IP5):
        if (pkt_tcp.flags & 2): # SYN
            actions = [parser.OFPActionOutput(3)]
        else:
            actions = [ ]
        match = parser.OFPMatch(eth_type=ETH_TYPE_IP, ipv4_dst=IP5,
            ip_proto=IPPROTO_TCP, tcp_dst=pkt_tcp.dst_port)
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
            match=match, instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)
        return

    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=buffer_id,
        in_port=in_port, actions=actions)
    datapath.send_msg(out)
```

L'host h6 porta 1234 apre una connessione TCP verso h1 porta 80.

(a) (2 points) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s1$: MAC6 → MAC1; IPv4; IP6 → IP1; 198; TCP; 1234 → 8080

(b) (1 point) L'host h1 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s3$: MAC1 → MAC6; IPv4; IP1 → IP6; 198; TCP; 80 → 1234

L'host h2 porta 8888 invia un datagramma UDP verso h6 porta 123.

(c) (1 point) Scrivere i messaggi dallo switch s2 al controllore.

Soluzione: Dipende dalla lunghezza del pacchetto. Se il pacchetto IP è grande ($\geq$ 114 ottetti) viene bloccato in s1. Altrimenti:
$s2 \rightarrow \mathcal{C}$: Packet_in(MAC2 → MAC6; IPv4; IP2 → IP6; 197; UDP; 8889 → 123)

(d) (1 point) Scrivere i messaggi dal controllore a s2.

Soluzione:
$\mathcal{C}$ → s2 : Packet_out(MAC2 → MAC6; IPv4; IP2 → IP6; 194; UDP; 8890 → 123, Output(FLOOD))

(e) (1 point) Scrivere i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
s2 → s3, h3, h4 : MAC2 → MAC6; IPv4; IP2 → IP6; 194; UDP; 8890 → 123

L'host h2 porta 7777 apre una connessione TCP verso l'host h5 porta 245.

(f) (1 point) Scrivere i messaggi dallo switch s2 al controllore.

Soluzione:
$s2 \rightarrow \mathcal{C}$: Packet_in(MAC2 → MAC5; IPv4; IP2 → IP5; 200; TCP; 7777 → 245)

(g) (1 point) Scrivere i messaggi dal controllore a s2.

Soluzione:
$\mathcal{C} \rightarrow s2$: Flow_Mod(priority=3,
datapath=s2,
match=(eth_type=ETH_TYPE_IP, ipv4_dst=IP5,
ip_proto=IPPROTO_TCP, tcp_dst=245),
buffer_id=...,
instructions=(APPLY_ACTIONS, Output(3))

(h) (1 point) Scrivere i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s3$: MAC2 → MAC5; IPv4; IP2 → IP5; 200; TCP; 7777 → 245

(i) (1 point) Scrivere il contenuto finale delle tabelle openflow.

Soluzione:

Viene aggiunta in s2 la seguente regola:

Switch s2		
Priority	Match	Action
5	eth_type = ETH_TYPE_IP ipv4_dst = IP5 ip_proto = IPPROTO_TCP tcp_dst = 245	Output(3)

Esercizio 2

Una matrice di commutazione crossbar di uno switch 4×4 usa l'algoritmo Parallel Iterative Matching (PIM) per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza fissata.

L'algoritmo di assegnazione dei ticket privilegia le interfacce di ingresso secondo la seguente regola:

- le interfacce di uscita provilegiano le interfacce di ingresso con l'identificativo minore
- le interfacce di ingresso privilegiano le interfacce di uscita con l'identificativo maggiore

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

Ingresso 1: 4, 2, 1

Ingresso 2: 1, 2, 2

Ingresso 3: –, 4, 3

Ingresso 4: –, 4, 1

(a) (2 points) Svolgere l'algoritmo PIM fino allo svuotamento di tutte le code, specificando ad ogni round quali grant sono richiesti ed assegnati e quali pacchetti sono trasmessi.

Soluzione:

Round		Ingresso 1	2	3	4
1	Richiesta	{1,2,4}	{1,2}	{3,4}	{1,4}
	Assegnazione	{1,2,4}	–	{3}	–
	Trasmissione	4	–	3	–
2	Richiesta	{1,2}	{1,2}	{4}	{1,4}
	Assegnazione	{1,2}	–	{4}	–
	Trasmissione	2	–	4	–
3	Richiesta	{1}	{1,2}	–	{1,4}
	Assegnazione	{1}	{2}	–	{4}
	Trasmissione	1	2	–	4
4	Richiesta	–	{1,2}	–	{1}
	Assegnazione	–	{1,2}	–	-
	Trasmissione	–	2	–	–
5	Richiesta	–	{1}	–	{1}
	Assegnazione	–	{1}	–	-
	Trasmissione	–	1	–	–
6	Richiesta	–	–	–	{1}
	Assegnazione	–	–	–	{1}
	Trasmissione	–	–	–	1

(b) (2 points) Si consideri il caso in cui al termine del secondo round si presenti un pacchetto alla coda 3 diretto all'uscita 3 e un pacchetto alla coda 4 diretto all'uscita 2. Ripetere la simulazione dell'algoritmo PIM.

Soluzione:

Round		Ingresso 1	2	3	4
1	Richiesta	{1,2,4}	{1,2}	{3,4}	{1,4}
	Assegnazione	{1,2,4}	–	{3}	–
	Trasmissione	4	–	3	–
2	Richiesta	{1,2}	{1,2}	{4}	{1,4}
	Assegnazione	{1,2}	–	{4}	–
	Trasmissione	2	–	4	–
3	Richiesta	{1}	{1,2}	{3}-	{1,2,4}
	Assegnazione	{1}	{2}	{3}	{4}
	Trasmissione	1	2	3	4
4	Richiesta	–	{1,2}	–	{1,2}
	Assegnazione	–	{1,2}	–	-
	Trasmissione	–	2	–	–
5	Richiesta	–	{1}	–	{1,2}
	Assegnazione	–	{1}	–	{2}
	Trasmissione	–	1	–	2
6	Richiesta	–	–	–	{1}
	Assegnazione	–	–	–	{1}
	Trasmissione	–	–	–	1

Facendo riferimento all'algoritmo PIM con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature.

(c) (2 points) Qual è la probabilità che l'uscita numero 3 assegni un grant all'ingresso 1?

Soluzione:

$$p = \frac{1}{4} = 25\%$$

(d) (2 points) Qual è la probabilità che l'ingresso 1 riceva esattamente un grant?

Soluzione:

$$p = \binom{4}{1}\left(\frac{3}{4}\right)^3 \frac{1}{4} \simeq 42\%$$

(e) (1 point) In quale caso tutte le uscite sono usate? Qual è la probabilità che ciò accada?

Soluzione: Le uscite assegnano grant a ingressi tutti diversi.

$$p = \frac{4!}{4^4} \simeq 9\%$$

(f) (1 point) In quale caso è usata una sola uscita? Qual è la probabilità che ciò accada?

Soluzione: Le uscite assegnano grant allo stesso ingresso.

$$p = \frac{4}{4^4} \simeq 1{,}5\%$$

Esercizio 3

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione

è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1: 50, 150, 150

Coda 2: 225, 50, 50

Coda 3: 100, 150, 100

Coda 4: 50

(a) (4 points) Si considerino quanti di lunghezza 100. Simulare l'algoritmo DRR, indicando il tempo al quale termina ogni trasmissione ed il deficit residuo.

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Pacchetto trasmesso	Tempo fine tx
1	1	0	100	–	–
	2	0	0	50 + 50	100
	3	0	0	100	200
	4	0	0	50	250
2	1	100	50	150	400
	2	0	100	–	–
	3	0	100	–	–
3	1	50	0	150	550
	2	100	200	–	–
	3	100	50	150	700
4	1	0	0	50	750
	2	200	0	225	975
	3	50	0	100	1075

(b) (3 points) Ripetere la simulazione considerando che:

- le code 1 e 2 hanno quanti di lunghezza 200;
- le code 3 e 4 hanno quanti di lunghezza 100;
- al tempo $t = 400$ ns si presenta alla coda 4 un pacchetto di lunghezza 100.

Soluzione:

Round	Coda	Deficit iniziale	Deficit finale	Pacchetto trasmesso	Tempo fine tx
1	1	0	50	150	150
	2	0	100	50 + 50	250
	3	0	0	100	350
	4	0	0	50	400
2	1	50	0	150+50	600
	2	100	0	225	825
	3	100	50	150	975
	4	0	0	100	1075
3	3	50	0	100	1175

(c) (3 points) Si consideri uno scenario con lunghezza massima dei pacchetti 1000 byte. Calcolare il valore minimo del quanto che garantisce che le code 2, 3 e 4 abbiano collettivamente la stessa banda della coda 1.

Soluzione: Dobbiamo rispettare il vincolo $q_2 + q_3 + q_4 = q_1$. Qualsiasi soluzione con $q_2 = q_3 = q_4$ e con $q_1 = 3q_2$ va bene. Per trasmettere almeno un pacchetto per ogni coda ad ogni round fissiamo $q_2 = q_3 = q_4 = 1000$ e $q_1 = 3000$.

Esercizio 4

Si consideri un sistema di misura del traffico in un router. Ai pacchetti entranti da tutti i link è associato un identificativo di flusso x ottenuto concatenando alcuni campi del pacchetto. La lunghezza massima dei pacchetti è 1024 byte.

Sull'identificativo x sono calcolati n hash usando la famiglia di funzioni indipendenti $h_i(x)$ con $1 \leq i \leq n$. Tali hash sono usati come indici in n array distinti $A_1, \ldots, A_n$ di m elementi ciascuno. Ad ogni pacchetto arrivato la $h_i(x)$-esima posizione di ognuno del i-esimo array per tutti gli i è incrementata di un valore pari alla lunghezza del pacchetto.

Se $A_i[h_i(x)] \geq S$ per ogni i, l'indentificativo x viene salvato su disco.

Tutti gli array sono azzerati ogni $T = 1\,\text{h}$.

(a) (2 points) Considerando 6 collegamenti bidirezionali con capacità $C = 1\,\text{Gbit/s}$, dire quanti bit occorrono per rappresentare gli n array.

Soluzione: Nel caso peggiore, in un secondo si accumulano $6 \cdot 10^9 \cdot 3600$ bit in un unico contatore. Contando in ottetti, servono

$$\lceil \log_2 6 \cdot 10^9 / 8 \cdot 3600 \rceil mn = 42mn \text{ bit}$$

(b) (2 points) Dire quali campi occorre scegliere per rappresentare l'identificativo del flusso per avere nello stesso flusso tutti i pacchetti tra la stessa coppia di host in entrambe le direzioni.

Soluzione:

$$x = \min(\text{IPs,IPd}) \| \max(\text{IPs,IPd})$$

(c) (2 points) Dire quali campi occorre scegliere per rappresentare l'identificativo del flusso per avere nello stesso flusso tutti i pacchetti verso lo stesso socket TCP lato server.

Soluzione:

$$x = \text{IPd} \| \text{ip_proto} \| \text{port_dst}$$

(d) (2 points) Dire come scegliere la soglia S in modo che si scriva su disco l'identificativo di tutti i flussi di almeno 25 Mbit/s. E' possibile che ci siano falsi positivi o falsi negativi?

Soluzione: La soglia va fissata a $S = 25\,\text{Mbit/s} \cdot 3\,600\,\text{s}/8 = 11{,}25 \cdot 10^9$ ottetti. Ci possono essere falsi positivi, non ci sono falsi negativi.

(e) (1 point) Si consideri un unico flusso y attivo nel sistema, che ha superato i 25 Mbit/s nell'attuale intervallo di misura. Considerando $m = 2^8$ e $n = 3$, dire qual è la probabilità che il primo pacchetto di un nuovo flusso sia copiato su disco.

Soluzione: Si ha un falso positivo se il secondo flusso x collide con y su tutti gli hash. Questo accade con probabilità:

$$\left(\frac{1}{m}\right)^n = \frac{1}{2^{24}} = 6 \cdot 10^{-8}$$

(f) (1 point) Si considerino 64 flussi attivi nel sistema, che hanno tutti superato i 25 Mbit/s nell'attuale intervallo di misura. Considerando $m = 2^8$ e $n = 3$, dire qual è la probabilità che il primo pacchetto di un nuovo flusso sia copiato su disco. Supporre che non vi siano collisioni tra le funzioni hash calcolate sui 64 flussi già attivi.

Soluzione: Si ha un falso positivo se il nuovo flusso x collide con un qualche altro flusso su tutti gli hash. Questo accade con probabilità:

$$\left(\frac{64}{m}\right)^n = \frac{1}{64} = 0{,}015\,6$$